

Emalog — o que foi feito em 17 de setembro de 2026

Central de Atendimento no WhatsApp, banco de produção no RDS e servidor próprio da Evolution.
Documento para retomar o trabalho numa conversa nova.

Resumo

- **Central de Atendimento pronta, Fases 0 a 4**, publicada e conferida no ar: fila compartilhada entre atendentes, leitura, avisos, busca, anexos e relatórios.
- **O painel de conversas passou a viver dentro da Central de Contratação**, na aba Conversas, no lugar do Chatwoot, que saiu do sistema.
- **Descoberto um problema grave**: a produção gravava num banco SQLite dentro da própria aplicação e perdia tudo a cada deploy. Sempre foi assim, desde antes da Central.
- **Banco PostgreSQL criado na AWS** (RDS em log-db) e ligado ao sistema. O diagnóstico já mostra "dialeto": "postgresql".
- **Servidor próprio da Evolution no ar**, num EC2 da sua conta, no lugar do servidor de terceiros que caiu e não voltou.
- **Dois credenciais vazaram em prints** enviados pela conversa e precisam ser trocadas antes de qualquer outra coisa: a senha do banco e a chave da Evolution.

1. Central de Atendimento no WhatsApp

Módulo atendimento_conversas/, feito fase por fase, cada uma com testes que rodam em SQLite e em PostgreSQL.

0	Conversas, histórico de eventos e anexos no banco. Mensagens antigas passaram a ter conversa e marca de leitura.
1	Entrada e saída de mensagens. O recebimento pela Twilio valida assinatura; o envio sai pela Evolution, porque a conta Twilio é de teste e não envia texto livre.
2	Fila compartilhada: assumir, transferir, liberar, devolver ao bot e resolver. Dois atendentes clicando ao mesmo tempo é decidido no banco, não no navegador.
3	Marcação de lida, aviso de mensagem nova em qualquer tela, busca por nome ou telefone e anexos guardados no S3.
4	Relatórios para administrador: volume por dia e semana, tempo de resposta por atendente, esperas sem resposta e exportação em CSV.
5	Chatbot de regras. Não começou. Só começa quando as fases anteriores estiverem estáveis em produção.

Painel dentro da Central de Contratação

A aba Conversas da Contratação abria o Chatwoot numa janela. Agora ela mostra a própria Central, para atender junto do Kanban. O painel virou um arquivo só, usado pelas duas telas, e os botões de som, avisos, atualizar e relatórios ficam dentro dele. O acesso ao Chatwoot foi removido do sistema.

Trava contra o banco errado

Se o PostgreSQL não responder no boot, a aplicação caía para um SQLite vazio sem avisar ninguém. Agora, quando está na AWS, ela tenta o PostgreSQL por cerca de 50 segundos e se recusa a subir sem

ele. Essa proteção ainda não foi publicada.

2. Infraestrutura criada na AWS

Banco de dados

- PostgreSQL em `log-db`, no RDS, na região de São Paulo, sem acesso pela internet: só o servidor do sistema alcança.
- Criptografado, com proteção contra exclusão acidental e backup automático.
- Criado pelo script `infra/aws/criar_banco_rds.sh`, que roda no AWS CloudShell e pode ser rodado de novo sem duplicar nada.
- O banco começou vazio. O que estava no SQLite antigo não foi copiado.

Servidor da Evolution

- O servidor antigo, de terceiros, saiu do ar e não voltou. O dono dele tinha acesso à sessão do WhatsApp da empresa.
- Agora existe um servidor próprio, o EC2 em `log-evolution`, com a Evolution 2.4.0, dados no mesmo banco e acesso liberado só para o sistema.
- Criado pelo script `infra/aws/criar_evolution_ec2.sh`. A conta da Evolution Foundation foi ativada e o painel foi fechado depois do uso.

3. O que falta fazer, em ordem

- 1 **Trocar a senha do banco.** Ela apareceu inteira num print enviado pela conversa. A troca é pelo CloudShell, e depois a linha nova vai para o arquivo `.env`.
- 2 **Trocar a chave da Evolution**, que vazou do mesmo jeito. Fazer isso antes de escanear o QR code.
- 3 **Escanear o QR code** em Central de contratação, Cadastros EMA, botão Sincronizar WhatsApp. Antes disso, remover do celular da empresa toda sessão desconhecida em Aparelhos conectados.
- 4 **Publicar o pacote com a trava contra SQLite** e conferir em `/conversas/api/status` se aparece `"trava_sqlite": true`.
- 5 **Salvar no histórico do projeto o trabalho que ainda não foi salvo:** o painel dentro da Contratação, a trava contra SQLite, o script da Evolution e as pendências atualizadas.
- 6 **Trocar as senhas e chaves antigas** que estão no histórico do projeto e as que passaram por conversa, incluindo o token da Twilio.
- 7 **Instalar o certificado de segurança (HTTPS).** Hoje as conversas com motorista, inclusive documentos, trafegam em texto aberto.
- 8 **Configurar o armazenamento de arquivos (S3)**, que hoje está com um valor de exemplo e quebra o envio de documento de motorista.
- 9 **Só então começar a Fase 5**, o chatbot de regras.

A lista completa, com o detalhe técnico de cada item, está no arquivo `atendimento_conversas/PENDENCIAS.md`, dentro do projeto.

4. Onde está cada coisa

Central de Atendimento	atendimento_conversas/
Plano das fases	atendimento_conversas/PLANO_DE_FASES.md
Pendências	atendimento_conversas/PENDENCIAS.md
Scripts da AWS	infra/aws/
Testes	testes/, com um LEIAME em cada pasta, explicando o que prova e como testar à mão
Pacotes de deploy	FONTE AWS RODANDO/
Senhas e chaves	.env, na raiz do projeto, fora do controle de versão

5. Como o trabalho foi conduzido

- Uma fase por vez, com parada e relatório ao fim de cada uma.
- Toda alteração de banco testada em SQLite e em PostgreSQL, sem comando que só funcione em um deles.
- Nenhuma senha dentro do código: tudo por variável de ambiente, guardado no arquivo .env.
- Defeito só é afirmado depois de reproduzir em teste. Vários testes foram checados de propósito com o código quebrado, para provar que eles pegam o erro.
- Os scripts da AWS foram testados contra uma AWS simulada antes de rodarem de verdade.

Para retomar numa conversa nova, basta pedir a continuação: o assistente já guarda esse contexto em memória e lê as pendências do projeto antes de propor os próximos passos.